



Devoir 1

Algorithmique sur les Graphes et Combinatoire

Master 1 Informatique Avancée et Application (I2A)

Université Marie & Louis Pasteur, Franche-Comté

N'jie Zamon

Formation à distance en alternance
Depuis la Martinique

Novembre 2025



Table des matières

1	Exercice 1 : Graphe partiel sans suivant (9 points)	2
1.1	Question 1.1 : Savoir si un graphe ne contient pas d'arc suivant	2
1.1.1	1.a : Présenter une solution	2
1.1.2	1.b : Donner une fonction <code>sansSuivant(FSP, APSP)</code>	2
1.1.3	1.c : Dérouler la fonction	3
1.2	Question 1.2 : Algorithme de recherche d'un graphe partiel maximal sans suivant	3
1.2.1	2.a : Donner un graphe partiel maximal sans arc suivant du graphe G (figure 1)	3
1.2.2	2.b : Présenter une solution pour la recherche	4
1.2.3	2.c : Donner une fonction <code>partielSansSuivant(FS, APS)</code>	4
1.2.4	2.d : Dérouler votre algorithme sur le graphe de la figure 1	5
2	Exercice 2 : Recherche des isthmes d'un graphe (6 points)	5
2.1	Question 2.1 : Savoir si l'arête $\{x, y\}$ est un isthme	6
2.1.1	1.a : Présenter une solution générique	6
2.1.2	1.b : Donner une fonction <code>estUnIsthme(x, y, M)</code>	6
2.2	Question 2.2 : Obtenir la liste des isthmes du graphe G	7
2.2.1	2.a : Présenter une solution générique	7
2.2.2	2.b : Donner une fonction <code>listeIsthmes(M)</code> - Version naïve	7
2.2.3	2.c : Donner une fonction <code>listeIsthmes(M)</code> - Version optimisée (Tarjan) .	7
3	Exercice 3 : Confection de gâteaux (5 points)	8
3.1	Question 3.1 : Écrire le programme linéaire	9
3.1.1	Variables de décision	9
3.1.2	Objectif à maximiser	9
3.1.3	Contraintes	9
3.2	Question 3.2 : Simplifier le problème	9
3.3	Question 3.3 : Résoudre le problème avec la méthode graphique	10
3.4	Question 3.4 : Résoudre le problème avec la méthode du simplexe	10
3.4.1	Itération 1	11
3.4.2	Itération 2	11
3.4.3	Itération 3 (Finale)	11

1 EXERCICE 1 : GRAPHE PARTIEL SANS SUIVANT (9 POINTS)

1 Exercice 1 : Graphe partiel sans suivant (9 points)

Nous rappelons qu'un arc (w, x) suit l'arc (u, v) si $v = w$. Un graphe partiel sans suivant ne contient donc pas de chemin de longueur 2.

1.1 Question 1.1 : Savoir si un graphe ne contient pas d'arc suivant

1.1.1 1.a : Présenter une solution

Principe de la solution

Un graphe partiel $GP = (V, E_P)$ ne contient pas d'arc suivant si et seulement si aucun sommet $v \in V$ n'est à la fois une **tête d'arc** et une **queue d'arc** dans E_P . La solution consiste à vérifier, pour chaque sommet v :

- (i) Si v a un prédécesseur (est une tête d'arc) dans E_P .
- (ii) Si v a un successeur (est une queue d'arc) dans E_P .

Si un sommet v vérifie (i) ET (ii), alors l'arc (u, v) est suivi par (v, x) (ou inversement), et le graphe contient un arc suivant. Sinon, il n'en contient pas.

1.1.2 1.b : Donner une fonction sansSuivant(FSP, APSP)

Nous exploitons les tableaux FSP (File des Successeurs Partielle) et APSP (Adresse des Premiers Successeurs Partielle).

```

1 Fonction sansSuivant(FSP, APSP) : Bool en
2   N = taille(APSP) - 1 // Nombre de sommets V
3
4   // Etape 1: Determiner les sommets qui sont des tetes d'arcs (ont un
5   // predecesseur)
6   aUnPredecesseur[0..N-1] = Faux
7
8   // Parcourir FSP: tout sommet v dans FSP est une tete d'arc
9   Pour i de 0 a taille(FSP)-1 faire
10    v = FSP[i]
11    aUnPredecesseur[v] = Vrai
12  Fin Pour
13
14  // Etape 2: Verifier si un sommet est a la fois tete ET queue d'arc
15  Pour v de 0 a N-1 faire
16    // Test si v a un successeur (est une queue d'arc)
17    // La condition est que l'intervalle de successeurs [APSP[v], APSP[v
18    +1]-1] ne soit pas vide.
19    Si (v < N-1) alors
20      aUnSuccesseur = (APSP[v] < APSP[v+1])
21    Sinon // Cas du dernier sommet
22      aUnSuccesseur = (APSP[v] < taille(FSP))
23    Fin Si
24
25    // Si v est tete d'un arc (predecesseur) ET queue d'un arc (successeur
26    )
27    Si aUnPredecesseur[v] ET aUnSuccesseur alors
28      Retourner Faux // Arc suivant trouve
29    Fin Si
30  Fin Pour
31
32  Retourner Vrai // Aucun arc suivant trouve
33 Fin Fonction

```

Listing 1 – Fonction sansSuivant

1 EXERCICE 1 : GRAPHE PARTIEL SANS SUIVANT (9 POINTS)

1.1.3 1.c : Dérouler la fonction

Trace d'exécution

Données : 'FSP = [2, 6, 2, 7, 5, 5, 7]', 'APSP = [0, 3, 6, 7, 9, 11, 13, 14]'

Étape 1 : Initialisation. 'N = 8'. 'aUnPredecesseur' est un tableau de 8 booléens initialisé à 'Faux'.

Étape 2 : Parcours de FSP pour trouver les têtes.

- On lit 'FSP[0]=2' -> 'aUnPredecesseur[2] = Vrai'.
- On lit 'FSP[1]=6' -> 'aUnPredecesseur[6] = Vrai'.
- On lit 'FSP[2]=2' -> 'aUnPredecesseur[2] = Vrai' (déjà Vrai).
- On lit 'FSP[3]=7' -> 'aUnPredecesseur[7] = Vrai'.
- On lit 'FSP[4]=5' -> 'aUnPredecesseur[5] = Vrai'.
- On lit 'FSP[5]=5' -> 'aUnPredecesseur[5] = Vrai' (déjà Vrai).
- On lit 'FSP[6]=7' -> 'aUnPredecesseur[7] = Vrai' (déjà Vrai).

À la fin : 'aUnPredecesseur = [F, F, V, F, V, V, V, V]'

Étape 3 : Parcours des sommets 'v' de 0 à 7.

- 'v = 0' : 'aUnPredecesseur[0]' est 'Faux'. On continue.
- 'v = 1' : 'aUnPredecesseur[1]' est 'Faux'. On continue.
- 'v = 2' : 'aUnPredecesseur[2]' est 'Vrai'. 'a-t-il un successeur ?' 'APSP[2]=6', 'APSP[3]=7'. '6 < 7' est 'Vrai'. Donc 'v=2' est à la fois tête et queue. **La fonction retourne 'Faux'.**

Conclusion : Le graphe partiel contient un arc suivant (par exemple, (0,2) est suivi de (2,3)).

1.2 Question 1.2 : Algorithme de recherche d'un graphe partiel maximal sans suivant

1.2.1 2.a : Donner un graphe partiel maximal sans arc suivant du graphe G (figure 1)

Le graphe partiel est $GP = (V, E_P)$, où E_P est un ensemble maximal d'arcs. Un ensemble maximal d'arcs sans suivant est :

$$E_P = \{(0, 2), (1, 7), (3, 5), (4, 7)\}$$

- **Têtes (H_P) :** {2, 7, 5}.
- **Queues (T_P) :** {0, 1, 3, 4}.
- **Intersection :** $H_P \cap T_P = \emptyset$.

Ce graphe partiel est maximal, car ajouter n'importe quel arc restant de G créerait un conflit (un sommet deviendrait à la fois tête et queue).

1 EXERCICE 1 : GRAPHE PARTIEL SANS SUIVANT (9 POINTS)

1.2.2 2.b : Présenter une solution pour la recherche

Heuristique gloutonne

La solution utilise une **heuristique gloutonne** pour construire un ensemble maximal d'arcs sans suivant.

1. **Initialisation** : L'ensemble d'arcs choisis E_P est vide. Deux ensembles de sommets sont créés : T_P (sommets de départ des arcs dans E_P) et H_P (sommets d'arrivée des arcs dans E_P).
2. **Parcours** : Parcourir les arcs (u, v) du graphe original $G(V, E)$ dans un ordre arbitraire (par exemple, par indice de sommet u).
3. **Règle de sélection** : Ajouter l'arc (u, v) à E_P si et seulement si $u \notin H_P$ et $v \notin T_P$.
4. **Mise à jour** : Si l'arc est ajouté, mettre à jour les ensembles $T_P \leftarrow T_P \cup \{u\}$ et $H_P \leftarrow H_P \cup \{v\}$.

Justification de la propriété maximale : L'algorithme garantit la maximalité. Pour tout arc $(u, v) \in E \setminus E_P$ qui n'a pas été choisi, c'est que soit $u \in H_P$ (créant $(w, u) \rightarrow (u, v)$), soit $v \in T_P$ (créant $(u, v) \rightarrow (v, x)$). Dans tous les cas, l'ajout de cet arc aurait créé un arc suivant, donc E_P est maximal.

Remarque importante : L'ordre de parcours des arcs influence le graphe partiel maximal obtenu. Différents ordres peuvent conduire à des graphes partiels maximaux distincts, mais tous respectent la propriété de ne pas contenir d'arc suivant et d'être maximaux.

1.2.3 2.c : Donner une fonction partielSansSuivant(FS, APS)

```

1  Fonction partielSansSuivant(FS, APS) : (FSP, APSP)
2      N = taille(APS) - 1 // Nombre de sommets
3
4      // T_P: Tableau booléen des sommets de d part (Queues) des arcs choisis
5      // H_P: Tableau bool en des sommets d'arriv e (Tetes) des arcs choisis
6      // Utilisation de tableaux pour une recherche en O(1)
7      Tails_P[0..N-1] = Faux
8      Heads_P[0..N-1] = Faux
9
10     // Initialisation des structures du Graphe Partiel (GP)
11     FSP = nouvelle_liste()
12     APSP[0..N]
13     index_FSP = 0
14
15     // Parcourir tous les sommets u du graphe G
16     Pour u de 0 a N-1 faire
17         APSP[u] = index_FSP // Adresse de depart des successeurs de u dans GP
18
19         // Parcourir les successeurs v de u dans G (via FS/APS)
20         Pour i de APS[u] a APS[u+1]-1 faire
21             v = FS[i]
22             // Conditions de s lection gloutonne avec tableaux booléens
23             Si (Heads_P[u] == Faux) ET (Tails_P[v] == Faux) alors
24                 // L'arc (u, v) est choisi
25                 Ajouter v a FSP
26
27             // Mettre a jour les ensembles interdits
28             Tails_P[u] = Vrai
29             Heads_P[v] = Vrai
30
31             index_FSP = index_FSP + 1
32     Fin Si

```

2 EXERCICE 2 : RECHERCHE DES ISTHMES D'UN GRAPHE (6 POINTS)

```

33     Fin Pour
34   Fin Pour
35
36   APSP[N] = index_FSP // Fin de la liste FSP
37
38   Retourner (FSP, APSP)
39 Fin Fonction

```

Listing 2 – Fonction partielSansSuivant

1.2.4 2.d : Dérouler votre algorithme sur le graphe de la figure 1

Graphe $G(V, E) : E = \{(0, 2), (0, 6), (1, 2), (1, 7), (2, 3), (2, 4), (3, 5), (4, 5), (4, 7), (5, 6), (5, 7), (6, 0)\}$.

u	v	Arc (u,v)	$u \in H_P ?$	$v \in T_P ?$	Ajouter ?	T_P (Queues)	H_P (Têtes)
0	2	(0,2)	Non	Non	Oui	{0}	{2}
0	6	(0,6)	Non	Non	Oui	{0}	{2, 6}
1	2	(1,2)	Non	Non	Oui	{0, 1}	{2, 6}
1	7	(1,7)	Non	Non	Oui	{0, 1}	{2, 6, 7}
2	3	(2,3)	Oui	Non	Non	-	-
2	4	(2,4)	Oui	Non	Non	-	-
3	5	(3,5)	Non	Non	Oui	{0, 1, 3}	{2, 6, 7, 5}
4	5	(4,5)	Non	Non	Oui	{0, 1, 3, 4}	{2, 6, 7, 5}
4	7	(4,7)	Non	Non	Oui	{0, 1, 3, 4}	{2, 6, 7, 5}
5	6	(5,6)	Oui	Non	Non	-	-
5	7	(5,7)	Oui	Non	Non	-	-
6	0	(6,0)	Oui	Oui	Non	-	-

Résultat

Résultat : $E_P = \{(0, 2), (0, 6), (1, 2), (1, 7), (3, 5), (4, 5), (4, 7)\}$ **Tableaux de sortie :**

- **FSP :** [2, 6, 2, 7, 5, 5, 7]
- **APSP :** [0, 2, 4, 4, 5, 7, 7, 7]

Conclusion de l'exercice 1 : Nous avons présenté une solution pour déterminer si un graphe partiel ne contient pas d'arc suivant, ainsi qu'un algorithme glouton pour construire un graphe partiel maximal sans arc suivant. La complexité de l'algorithme est en $O(|V| + |E|)$, ce qui est optimal pour ce problème.

2 Exercice 2 : Recherche des isthmes d'un graphe (6 points)

Un isthme est une arête dont la suppression augmente le nombre de composantes connexes. Nous considérons un graphe non orienté G connexe, donné par une matrice d'adjacence M .

2 EXERCICE 2 : RECHERCHE DES ISTHMES D'UN GRAPHE (6 POINTS)

2.1 Question 2.1 : Savoir si l'arête $\{x, y\}$ est un isthme

2.1.1 1.a : Présenter une solution générique

Principe de détection d'un isthme

Pour tester si l'arête $e = \{x, y\}$ est un isthme, nous appliquons la définition :

1. Créer un graphe temporaire G' en supprimant l'arête $\{x, y\}$ de G .
2. Effectuer un parcours (par exemple BFS ou DFS) dans G' en partant du sommet x .
3. Si le parcours atteint le sommet y , c'est qu'il existe un autre chemin entre x et y dans G' , donc $\{x, y\}$ **n'est pas** un isthme.
4. Si le parcours ne parvient pas à y , alors $\{x, y\}$ **est** un isthme.

2.1.2 1.b : Donner une fonction `estUnIsthme(x, y, M)`

```

1 Fonction estUnIsthme(x, y, M) : Bool en
2   N = taille(M) // Nombre de sommets
3
4   // Initialisation pour BFS
5   visite[0..N-1] = Faux
6   Queue = nouvelle_file()
7
8   visite[x] = Vrai
9   enfiler(Queue, x)
10
11  // Parcours en largeur (BFS) sans copie de matrice
12  Tant que Queue n'est pas vide faire
13    u = defiler(Queue)
14
15    // Parcourir les voisins v de u dans M
16    Pour v de 0 a N-1 faire
17      Si (M[u, v] = 1) alors
18        // Condition pour ignorer l'arete {x,y} dans les deux sens
19        Si ( (u == x ET v == y) OU (u == y ET v == x) ) alors
20          Continuer // On saute cette arete
21        Fin Si
22
23        Si v = y alors
24          Retourner Faux // Non isthme (y est atteint)
25        Fin Si
26
27        Si (visite[v] = Faux) alors
28          visite[v] = Vrai
29          enfiler(Queue, v)
30        Fin Si
31      Fin Si
32    Fin Pour
33  Fin Tant que
34
35  Retourner Vrai // Isthme (y n'a pas ete atteint)
36 Fin Fonction

```

Listing 3 – Fonction `estUnIsthme` (par BFS optimisé)

2 EXERCICE 2 : RECHERCHE DES ISTHMES D'UN GRAPHE (6 POINTS)

2.2 Question 2.2 : Obtenir la liste des isthmes du graphe G

2.2.1 2.a : Présenter une solution générique

Solution naïve pour la recherche des isthmes

Une approche naïve consiste à tester chaque arête du graphe pour déterminer si elle est un isthme en utilisant la fonction `estUnIsthme` définie précédemment.

1. Initialiser une liste vide L pour stocker les isthmes.
2. Pour chaque paire de sommets (i, j) avec $i < j$:
 - Si $M[i, j] = 1$ (il existe une arête entre i et j) :
 - Si `estUnIsthme(i, j, M)` retourne `Vrai`, ajouter l'arête $\{i, j\}$ à L .
3. Retourner la liste L .

Cette approche a une complexité temporelle de $O(|V| \cdot (|V| + |E|))$, car pour chaque arête (au plus $|V|^2$), nous effectuons un parcours du graphe en $O(|V| + |E|)$.

Algorithme de Tarjan pour les isthmes

La solution la plus performante est l'**Algorithme de Tarjan pour la recherche des ponts (isthmes)**. Elle est basée sur un unique parcours en profondeur (DFS) et opère en temps linéaire $O(|V| + |E|)$.

1. **Parcours DFS** : On effectue un DFS, en enregistrant pour chaque sommet u :
 - Le temps de découverte (`d[u]`).
 - Le plus bas temps de découverte accessible (`low[u]`) via des arcs de retour.
2. **Identification** : Pour une arête d'arbre $\{u, v\}$ (où u est le parent de v), si `low[v] > d[u]`, alors $\{u, v\}$ est un isthme, car il n'y a pas de chemin de retour de v ou de son sous-arbre vers u ou ses ancêtres.

2.2.2 2.b : Donner une fonction `listeIsthmes(M)` - Version naïve

```

1 Fonction listeIsthmes_Naive(M) : Liste
2   N = taille(M)
3   ListeIsthmes = nouvelle_liste()
4
5   // Parcourir toutes les paires de sommets (i,j) avec i < j
6   Pour i de 0 a N-2 faire
7     Pour j de i+1 a N-1 faire
8       Si M[i, j] = 1 alors // Si {i, j} est une arete
9         Si estUnIsthme(i, j, M) alors
10          Ajouter l'arete {i, j} a ListeIsthmes
11        Fin Si
12      Fin Si
13    Fin Pour
14  Fin Pour
15
16  Retourner ListeIsthmes
17 Fin Fonction

```

Listing 4 – Fonction `listeIsthmes` (Version naïve)2.2.3 2.c : Donner une fonction `listeIsthmes(M)` - Version optimisée (Tarjan)

3 EXERCICE 3 : CONFECTION DE GÂTEAUX (5 POINTS)

```

1 Fonction listeIsthmes(M) : Liste
2   N = taille(M)
3   d[0..N-1], low[0..N-1] // Tableaux de temps de decouverte et plus bas
   ancetre
4   timer = 0 // Variable globale ou passe par reference a la fonction
   r cursive
5   ListeIsthmes = nouvelle_liste()
6   visite[0..N-1] = Faux
7
8   Fonction DFS_Isthmus(u, parent)
9     timer = timer + 1
10    d[u] = timer
11    low[u] = timer
12    visite[u] = Vrai
13
14    // Parcourir les voisins v de u
15    Pour v de 0 a N-1 faire
16      Si M[u, v] = 1 alors // Si {u, v} est une arete
17        Si v = parent Alors
18          Continuer // Ignorer l'arete qui remonte vers le parent
19      Fin Si
20
21      Si visite[v] = Vrai Alors // Arete de retour (back edge)
22        low[u] = min(low[u], d[v])
23      Sinon // Arete d'arbre (tree edge)
24        DFS_Isthmus(v, u) // Appel recursif (v devient le fils)
25
26        // Mise a jour apr s le retour de la recursion
27        low[u] = min(low[u], low[v])
28
29        // Condition de l'isthme
30        Si low[v] > d[u] Alors
31          Ajouter l'arete {u, v} ListeIsthmes
32        Fin Si
33      Fin Si
34    Fin Pour
35  Fin Fonction
36
37  // Demarrer le DFS a partir du sommet 0 (graphe connexe)
38  DFS_Isthmus(0, -1)
39
40  Retourner ListeIsthmes
41 Fin Fonction

```

Listing 5 – Fonction listeIsthmes (Algorithme de Tarjan)

Conclusion de l'exercice 2 : Nous avons présenté deux approches pour la recherche des isthmes dans un graphe : une approche naïve de complexité $O(|V| \cdot (|V| + |E|))$ et une approche optimisée basée sur l'algorithme de Tarjan de complexité linéaire $O(|V| + |E|)$. L'algorithme de Tarjan est nettement plus performant pour les graphes de grande taille.

3 Exercice 3 : Confection de gâteaux (5 points)

Nous cherchons à maximiser le nombre de parts de gâteau en utilisant les ressources disponibles.

3 EXERCICE 3 : CONFECTION DE GÂTEAUX (5 POINTS)

3.1 Question 3.1 : Écrire le programme linéaire

3.1.1 Variables de décision

- x_1 : Nombre de petits gâteaux confectionnés ($x_1 \in \mathbb{N}$).
- x_2 : Nombre de grands gâteaux confectionnés ($x_2 \in \mathbb{N}$).

3.1.2 Objectif à maximiser

L'objectif est de maximiser le nombre total de parts de gâteau :

$$Z = 6x_1 + 10x_2$$

3.1.3 Contraintes

Les ressources disponibles doivent être converties en unités cohérentes (grammes).

- **Œufs** (7 disponibles) :

$$x_1 + x_2 \leq 7$$

- **Sucre** (1 kg = 1000 gr disponibles) :

$$100x_1 + 200x_2 \leq 1000$$

- **Farine** (1.8 kg = 1800 gr disponibles) :

$$200x_1 + 300x_2 \leq 1800$$

Contraintes de non-négativité :

$$x_1 \geq 0, \quad x_2 \geq 0$$

3.2 Question 3.2 : Simplifier le problème

Nous pouvons simplifier les inéquations en divisant tous les termes par un diviseur commun, sans affecter la région admissible.

$$\text{— Contrainte Sucre : } 100x_1 + 200x_2 \leq 1000 \quad \xrightarrow{\div 100} \quad x_1 + 2x_2 \leq 10$$

$$\text{— Contrainte Farine : } 200x_1 + 300x_2 \leq 1800 \quad \xrightarrow{\div 100} \quad 2x_1 + 3x_2 \leq 18$$

Programme Linéaire Simplifié

$$\begin{aligned} &\text{Maximiser } Z = 6x_1 + 10x_2 \\ &\text{sous contraintes} \\ &\quad (\text{L1}) : x_1 + x_2 \leq 7 \\ &\quad (\text{L2}) : x_1 + 2x_2 \leq 10 \\ &\quad (\text{L3}) : 2x_1 + 3x_2 \leq 18 \\ &\quad (\text{Non-Négativité}) : x_1 \geq 0, x_2 \geq 0 \end{aligned}$$

3 EXERCICE 3 : CONFECTION DE GÂTEAUX (5 POINTS)

3.3 Question 3.3 : Résoudre le problème avec la méthode graphique

La solution optimale se situe à un des sommets de la région admissible. Nous évaluons les points d'intersection des contraintes.

- **Intersection (L2) et (L3) :** $\begin{cases} x_1 + 2x_2 = 10 \\ 2x_1 + 3x_2 = 18 \end{cases} \implies (x_1, x_2) = (6, 2)$. Vérification de (L1) : $6 + 2 = 8 > 7$. Ce point est **inadmissible**.
- **Intersection (L1) et (L2) :** $\begin{cases} x_1 + x_2 = 7 \\ x_1 + 2x_2 = 10 \end{cases} \implies (x_1, x_2) = (4, 3)$. Vérification de (L3) : $2(4) + 3(3) = 17 \leq 18$. Le point $(4, 3)$ est **admissible**.

Évaluation des sommets admissibles :

Sommet	x_1	x_2	$Z = 6x_1 + 10x_2$
$(0, 0)$	0	0	0
$(0, 5)$ [Intersection L2 et $x_1 = 0$]	0	5	$6(0) + 10(5) = 50$
$(4, 3)$ [Intersection L1 et L2]	4	3	$6(4) + 10(3) = 54$
$(7, 0)$ [Intersection L1 et $x_2 = 0$]	7	0	$6(7) + 10(0) = 42$

La solution optimale est $x_1 = 4$ et $x_2 = 3$, avec $Z_{max} = 54$.

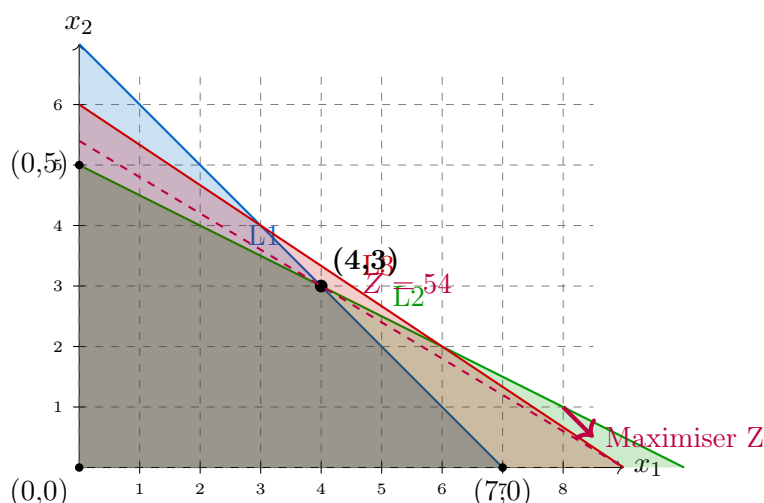


FIGURE 1 – Résolution graphique du problème de confection de gâteaux

3.4 Question 3.4 : Résoudre le problème avec la méthode du simplexe

Nous convertissons le programme linéaire en forme standard en ajoutant des variables d'écart $s_1, s_2, s_3 \geq 0$.

$$\text{Maximiser } Z = 6x_1 + 10x_2 + 0s_1 + 0s_2 + 0s_3$$

Sous contraintes

$$(C1) : x_1 + x_2 + s_1 = 7 \tag{1}$$

$$(C2) : x_1 + 2x_2 + s_2 = 10$$

$$(C3) : 2x_1 + 3x_2 + s_3 = 18$$

3 EXERCICE 3 : CONFECTION DE GÂTEAUX (5 POINTS)

3.4.1 Itération 1

Variable entrante : x_2 (coefficient le plus négatif : -10). Variable sortante : s_2 (ratio minimum : $10/2 = 5$). Pivot : 2.

Base	x_1	x_2	s_1	s_2	s_3	Cste	Ratio
s_1	1	1	1	0	0	7	$7/1 = 7$
s_2	1	2	0	1	0	10	$10/2 = 5$ (s_2 sort)
s_3	2	3	0	0	1	18	$18/3 = 6$
Z	-6	-10	0	0	0	0	

3.4.2 Itération 2

Après pivotage sur l'élément 2 (ligne s_2 , colonne x_2), on obtient le nouveau tableau. La variable entrante est x_1 (coefficient -1) et la sortante est s_1 (ratio minimum : $2/(1/2) = 4$).

Base	x_1	x_2	s_1	s_2	s_3	Cste	Ratio
s_1	$1/2$	0	1	$-1/2$	0	2	$2/(1/2) = 4$ (s_1 sort)
x_2	$1/2$	1	0	$1/2$	0	5	$5/(1/2) = 10$
s_3	$1/2$	0	0	$-3/2$	1	3	$3/(1/2) = 6$
Z	-1	0	0	5	0	50	

3.4.3 Itération 3 (Finale)

Après pivotage sur l'élément $1/2$ (ligne s_1 , colonne x_1), on obtient le tableau final.

Base	x_1	x_2	s_1	s_2	s_3	Cste
x_1	1	0	2	-1	0	4
x_2	0	1	-1	1	0	3
s_3	0	0	-1	-1	1	1
Z	0	0	2	4	0	54

Puisque tous les coefficients de la ligne Z sont ≥ 0 , la solution est optimale.

Solution finale

Solution : $x_1 = 4$ (petits gâteaux), $x_2 = 3$ (grands gâteaux), et $Z_{\max} = 54$ parts.

Conclusion de l'exercice 3 : Nous avons résolu un problème d'optimisation linéaire en utilisant deux méthodes différentes : la méthode graphique et la méthode du simplexe. Les deux méthodes convergent vers la même solution optimale : fabriquer 4 petits gâteaux et 3 grands gâteaux pour obtenir un maximum de 54 parts, tout en respectant les contraintes de ressources.